

CSV / JSON / OOP Project Review Sheet

Earlier books operated on data purely in memory — this one lets data "land" on disk: read it in from a file, crunch it, write it back out. CSV handles flat tables, JSON handles nesting, and a TodoManager mini-project stitches class, list-of-dict, and file storage into one complete pipeline.

Python 3

Tutor style

Includes diagrams

Includes FAQ

Includes mini-project

00 Review Order

File handling follows one fixed pipeline: **read** → **convert types** → **analyze** → **write back** → **re-read to verify**. Get CSV solid first — JSON is the same mindset in a different format — then merge both into the project.

01 ← start

Read CSV

DictReader

02

int/float
conversion

CSV is all strings

03

Analyze data

Reuse dict grouping

04

Write CSV

writer / writerow

05

JSON read/write

load / dump

06

TodoManager

OOP + persistence

One new friend running through the whole book: `with open(...) as file:`. Think of it as "borrowing a book" — you check the file out for a while, and the moment the `with` block ends, Python automatically "returns" it (closes the file) for you. Forgetting to close a file can leave data incompletely written; `with` has your back, so use it for every read and write.

01 CSV Basics + Reading

CSV = a plain-text table with comma-separated values. The first row is the header (column names), every row after that is one record. Suits data that's **uniformly structured, same fields on every row**.

student_scores.csv

name,subject,score

Alice,Math,85

Bob,Math,72

DictReader



```
{
  "name": "Alice",
  "subject": "Math",
  "score": "85"
}
```

← all strings!

```
score = int(row["score"]) # remember to convert
```

DictReader reads each row as a dict, but every value is str

read_scores — CSV → a list of records

csv read

import csv

def read_scores(filename):

records = []

with open(filename, "r") as file:

reader = csv.DictReader(file)

for row in reader:

name = row["name"]

subject = row["subject"]

score = int(row["score"])

records.append((name, subject, score))

return records

"r" = read mode; with auto-closes

feed it a file object, not a filename!

"85" → 85, or math won't work

Three things to lock in at once: ① `open(f, "r")` opens read mode; ② `csv.DictReader(file)` lets you access values by column name via `row["name"]`; ③ CSV is plain text — it has **no types**, every number reads back as a string, and you have to convert with `int()` / `float()` yourself.

Q I got `FileNotFoundError` but the file is right there — why?

Python resolves relative paths against **your terminal's current directory**, not the script's directory. Two fixes: ① `cd` into the data folder before running; ② pass a **full path** like `read_scores("~/cs/data/student_scores.csv")`. The more robust fix is wrapping it in `try / except` — that's coming in your next stage; for now just get clear on the path issue itself.

02 Analyzing CSV Data

Once data is read into records, analysis is back on familiar ground — **dict grouping + computation**, the exact template from your hash table book.

calculate_student_averages — group then average

analyze

```
def calculate_student_averages(records):
    grouped = {}
    for name, subject, score in records:
        if name not in grouped:           # the dict grouping template — remember this?
            grouped[name] = []
        grouped[name].append(score)

    averages = {}
    for name, scores in grouped.items():
        averages[name] = round(sum(scores) / len(scores), 2)
    return averages
```

find_failed_records — filter out failing scores

analyze

```
def find_failed_records(records, pass_score=70):
    failed = []
    for name, subject, score in records:
        if score < pass_score:
            failed.append((name, subject, score))
    return failed
```

Notice the continuity: file handling's real value isn't the file itself — it's feeding real data into algorithms you already know. `grouped` here is exactly the dict-grouping template from the hash table book; `pass_score=70` is a **default parameter** — use 70 if nothing's passed, or override it.

03 Writing CSV + Reading Back to Verify

Write analysis results back to a file with `csv.writer`, one row at a time via `writerow`. Then read it back to confirm nothing got written wrong.

write_averages — "w" write mode + writerow

csv write

```
def write_averages(filename, averages):
    with open(filename, "w", newline="") as file: # "w" overwrites; newline="" prevents blank lines
        writer = csv.writer(file)
        writer.writerow(["name", "average"])      # write the header row first
        for name, average in averages.items():
            writer.writerow([name, average])      # each record is one row (pass a list)
```

```
def read_averages(filename):
    averages = {}
    with open(filename, "r") as file:
        reader = csv.DictReader(file)
        for row in reader:
            averages[row["name"]] = float(row["average"])    # "85.0" → 85.0
    return averages
```

Two gotchas up front: ① **"w" overwrites** — if the file exists it gets wiped and rewritten, don't use it on something important carelessly (use **"a"** to append instead). ② Add **newline=""** when writing CSV, or some systems will insert extra blank lines between rows. The average read back is still a string — verify it with **float()** .

Empty data → only header written

Fields containing commas need quoting

"w" overwrites · "a" appends

04 JSON Basics

CSV can only express "flat tables." Once data has **nesting** (one student, multiple subjects and scores), reach for JSON — it maps naturally onto Python's dict / list.

JSON ↔ Python Mapping

JSON	PYTHON	JSON	PYTHON
object {}	dict	number	int / float
array []	list	true / false	True / False
string	str	null	None

The key difference: JSON numbers come back with their **type already intact** (genuine int/float), unlike CSV's all-strings — so after reading JSON you usually don't need to convert types. Watch three small traps: JSON uses **true/false/null** (lowercase), Python uses **True/False/None** (capitalized).

05 Reading & Analyzing Nested JSON

Read with `json.load(file)` to get a Python structure in one step. Before reading, check whether the outermost level is **[or {** — that determines whether you get a list or a dict.

students (list)

student (dict)

"name": "Alice"

"scores" (dict)

"Math": 85, "Physics": 91, ...

A list containing dicts, dicts containing another dict — dig in one layer at a time with `[key]`

```
import json

def read_students_json(filename):
    with open(filename, "r") as file:
        return json.load(file)           # one step to a Python structure

def calculate_json_averages(students):
    averages = {}
    for student in students:           # students is a list
        name = student["name"]
        scores = student["scores"]     # scores is a dict
        averages[name] = round(sum(scores.values()) / len(scores), 2)
    return averages
```

find_json_failed_students – record each person's failed subjects

json analyze

```
def find_json_failed_students(students, pass_score=70):
    failed_students = {}
    for student in students:
        name = student["name"]
        failed_subjects = []
        for subject, score in student["scores"].items(): # walk the nested dict
            if score < pass_score:
                failed_subjects.append(subject)
        if failed_subjects:               # only record if non-empty → {"Bob": ["Physics"]}
            failed_students[name] = failed_subjects
    return failed_students
```

The rhythm of nested traversal: the outer `for student in students` walks every student; the inner `for subject, score in scores.items()` walks that student's subjects. `if failed_subjects:` means "only record if the list is non-empty" — an empty list is falsy in Python, so you don't need to spell out `len(...) > 0`.

06 Writing JSON

Use `json.dump(data, file, indent=4)` to write a Python structure out as a JSON file. `indent=4` gives you an indented, human-readable file.

write_summary_json – assemble a summary, then write it out

json write

```
def write_summary_json(filename, averages, failed_students):
    summary = {
        "averages": averages,
        "failed_students": failed_students,
    }
    with open(filename, "w") as file:
        json.dump(summary, file, indent=4)   # indent=4: readable, good-looking
```

Memory hook: `load` / `dump` operate on **files** (they take a file argument); if you want to convert to/from a string in memory instead, use the s-versions, `loads` / `dumps` (string). For now, stick with the file versions — remember "dump = pour into a file."

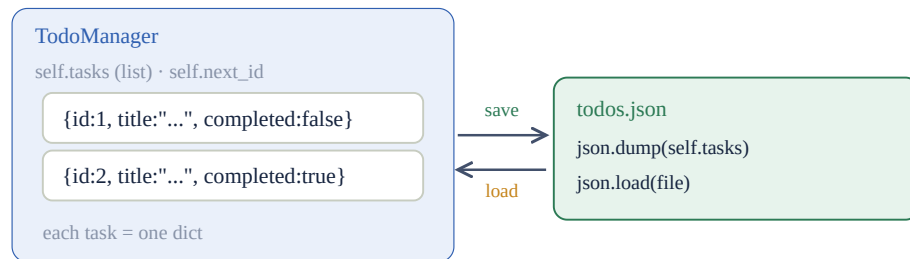
07 CSV vs JSON: Which One?

	BEST FOR	STRUCTURE	COMMON API
CSV	Flat tables, same fields on every row	2D table (rows × columns)	DictReader • writer • writerow
JSON	Nested / hierarchical data	dict inside list inside dict...	json.load • json.dump

One-sentence rule: if the data draws out as one tidy table → **CSV**; if the data draws out as a tree (nesting, optional fields) → **JSON**.
Also worth noting: CSV reads back all strings and needs manual type conversion, while JSON keeps its types automatically — one more small convenience in JSON's favor.

08 OOP + JSON Project: TodoManager

This merges everything so far: use a **class** to manage a **list of dict**, then use **JSON** to save/load it. This is a working little piece of software in miniature.



The object manages a list in memory; saving means dumping that list out as JSON

`__init__` + `add_task`

create

```
class TodoManager:
    def __init__(self):
        self.tasks = []           # store as instance attributes with self., not local variables!
        self.next_id = 1         # the first task starts at id=1

    def add_task(self, title):
        task = {"id": self.next_id, "title": title, "completed": False}
        self.tasks.append(task)
        self.next_id += 1         # key: increment right after using it, or ids will repeat
        return task
```

`complete_task` / `delete_task` — linear search by id

update

```
def complete_task(self, task_id):
    for task in self.tasks:      # walk the list looking for id (linear search)
        if task["id"] == task_id:
            task["completed"] = True
            return True
    return False                # not found

def delete_task(self, task_id):
    for index, task in enumerate(self.tasks):
        if task["id"] == task_id:
            self.tasks.pop(index) # index-based delete: intuitive while learning
            return True
    return False

def list_tasks(self):
    return self.tasks
```

```
def save_to_file(self, filename):
    with open(filename, "w") as file:
        json.dump(self.tasks, file, indent=4)

def load_from_file(self, filename):
    with open(filename, "r") as file:
        self.tasks = json.load(file)
    if not self.tasks:
        self.next_id = 1
    else:
        max_id = max(task["id"] for task in self.tasks)
        self.next_id = max_id + 1
```

Q How exactly should `self.next_id` be initialized, and why recompute it after loading?

Initialize with `self.next_id = 1` (the first task gets id 1). But after **loading** existing tasks from a file, id 2 might already be in use — if you don't update it, the next new task will grab id 2 again → an **id collision**. So after loading, recompute `next_id = max_id + 1`, continuing on from whatever the largest existing id is.

Q Why does it have to be `self.tasks` — why can't I just write tasks?

Without `self.`, `tasks` is just a **local variable** inside that one method — it disappears the moment the method returns, and no other method can see it. `self.tasks` is an **instance attribute** hanging off the object itself, shared across all of that object's methods and alive for its entire lifetime.

09 Bug Comparison Table

WRONG	WHY IT'S WRONG	CORRECT
<code>score + 10</code> (from a CSV read)	"85" is a string, can't do arithmetic on it	Convert first: <code>int(row["score"])</code>
<code>csv.DictReader(filename)</code>	Needs a file object, not a filename	First <code>with open(...) as file</code>
Writing CSV without <code>newline=""</code>	Extra blank lines on some systems	<code>open(f, "w", newline="")</code>
File not found	Relative path resolves against the wrong directory	<code>cd</code> into the right directory, or use a full path
<code>tasks = []</code> inside <code>__init__</code>	Just a local variable, doesn't persist	<code>self.tasks = []</code>
<code>add_task</code> forgets <code>next_id += 1</code>	Every task ends up with the same id	Increment after every append
Not recomputing <code>next_id</code> after load	New task ids collide with existing ones	<code>next_id = max_id + 1</code>
Using "w" when you meant to append	"w" wipes the entire file	Use "a" to append

Three rules of thumb to close out: ① CSV always reads back as strings — convert numbers with `int/float` before math (JSON doesn't need this). ② File operations always go through `with open(...) as file`, and readers need a **file object**, not a filename. ③ In OOP, state always lives on `self.`; id counters get "incremented right after use, recomputed right after loading."

10 Exercises

Ordered from easy to medium. Try treating the last group as one connected mini-project — that's file handling's real payoff.

◆ GROUP 1 • CSV (EASY)

- Easy** Read a scores CSV and print each record (remember to `int()` the score).
- Easy** Find the highest score per subject and write it to a new CSV.
- Medium** Read → compute per-person averages → write out → read back, and assert the two rounds of data match (a round-trip check).

◆ GROUP 2 • JSON (MEDIUM)

1. **Easy** Read students.json and print each student's name and subject count.
2. **Medium** Find the student with the highest average (nested traversal + comparison).
3. **Medium** Assemble the analysis into a summary dict, write it with `indent=4`, and open the file to check the formatting.

◆ GROUP 3 • TODOMANAGER EXTENSIONS (MEDIUM)

1. **Easy** Add `edit_task(task_id, new_title)`: rename a task by id.
2. **Medium** Add `list_pending()`: return only the incomplete tasks (filter on completed).
3. **Medium** Add a `priority` field to tasks, and sort output by priority.
4. **Medium** Run the full cycle: add a few tasks → save → build a new object → load → confirm next_id continues correctly.